Processor Prototyping Lab

Final Report

Nahom Tadesse

Zohaib Ali

Zhaoyu Jin

May 5, 2025

1 Executive Overview

The purpose of this report is to compare the performance and design trade-offs between three processor architectures: a pipelined processor without caches, a pipelined processor with a cache hierarchy, and a multicore processor with a cache hierarchy. Each design improves on the previous one by fixing issues which reduced performance such as latency and limited throughput. A cache works by frequently used data closer to the processor, allowing future accesses to avoid travelling all the way to the main memory. This significantly speeds up execution time. We used a split cache design, with one cache for instructions and another for data, to allow access at the same time to reduce average memory access time. This separation is needed to prevent structural hazards. The multicore design builds on these improvements by using parallelism, allowing multiple instructions to be done concurrently and increasing throughput.

To evaluate the effectiveness of these enhancements, we used the merge-sort benchmark program to gather consistent performance data across all designs. Metrics such as synthesis frequency, clock cycles per instruction (CPI), total execution time, instruction latency, FPGA resource utilization, and speedup were collected and compared. This report has the block and state diagrams for each design, an analysis of the performance results, and a discussion of the conclusions drawn from the project.

2 Processor Design

This section presents the block diagrams and state diagrams for the processor designs, caches, and bus controller used in the project.

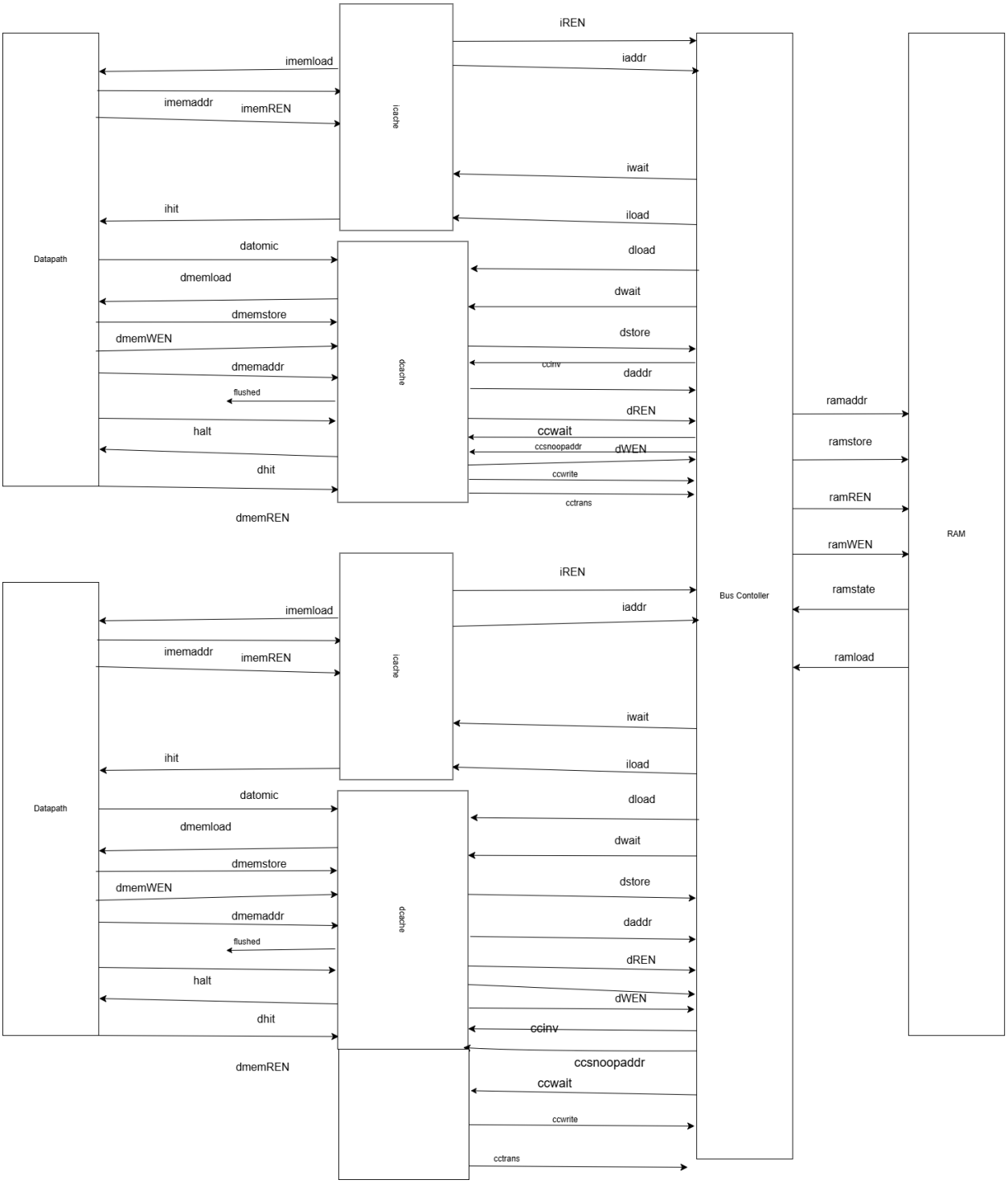

Figure 1: Overall Diagram

Figure 2 details the pipelined processor architecture with support for load-reserved and store-conditional instructions (LR/SC) with a reservation set. The diagram illustrates the standard pipeline stages, hazard detection units, forwarding paths, and LR/SC reservation tracking.

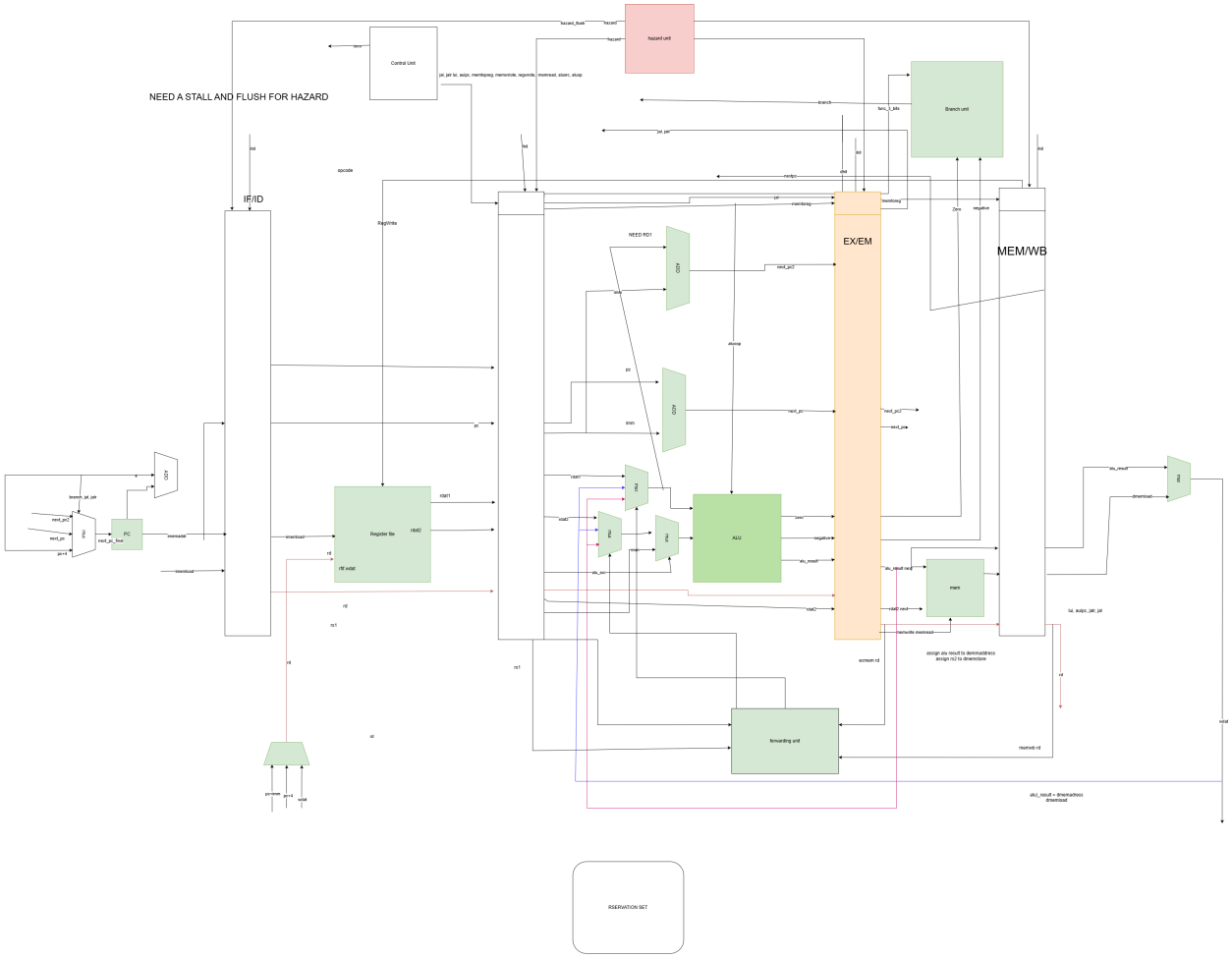

Figure 2: Pipelined Diagram with LRSC

Figure 3 shows the bus controller and the state diagram for it.

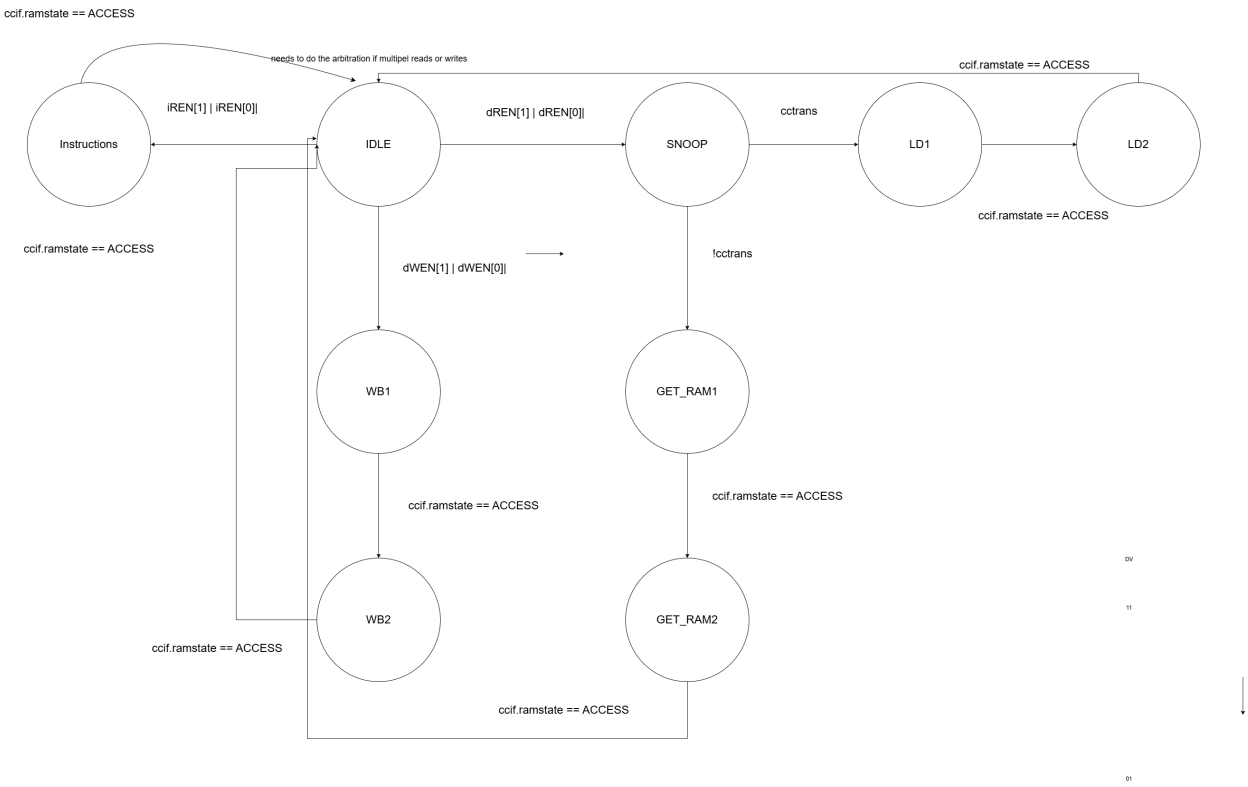

Figure 3: Bus Controller State Diagram

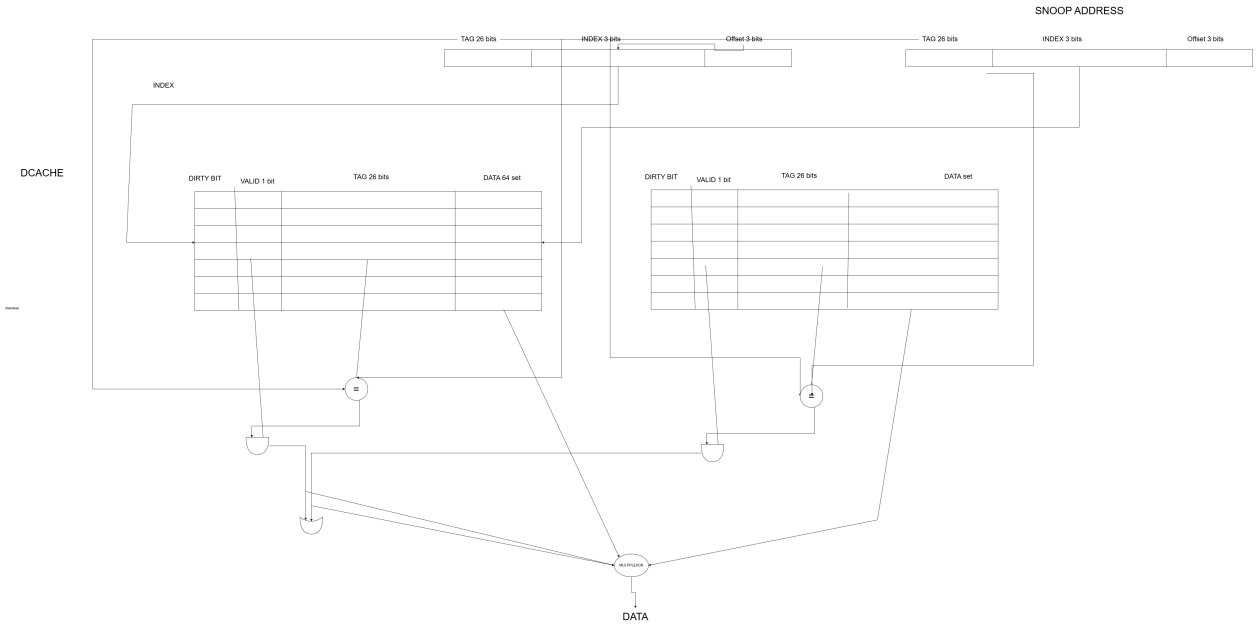

Figure 4: Data Cache Block Diagram

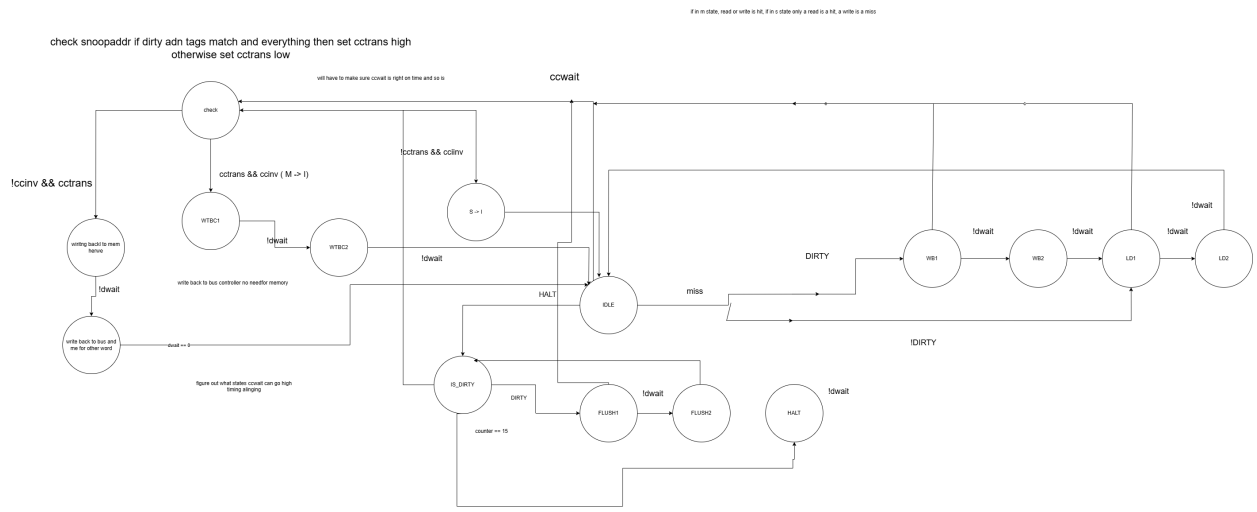

Figure 5: Data Cache State Diagram

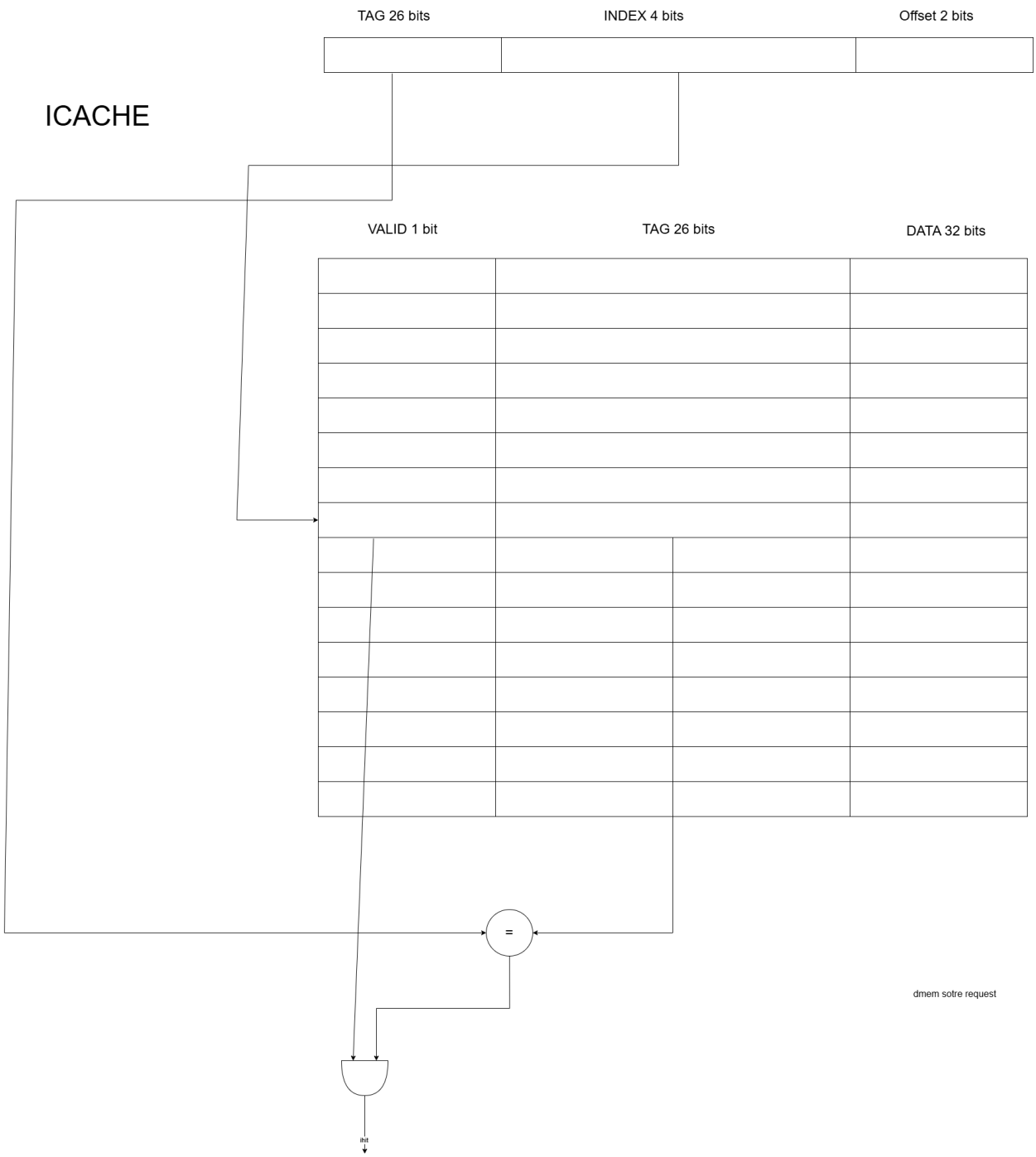

Figure 6: Instruction Cache Block Diagram

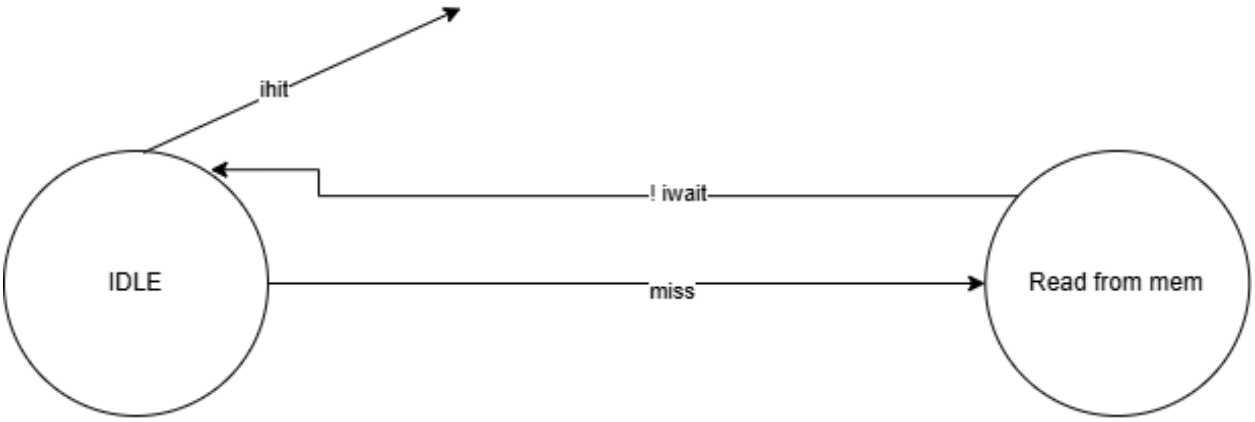

Figure 7: Instruction Cache State Diagram

3 Results

Metric (LAT = 6)	**Single Cycle**	**Pipeline**	**Caches + Pipeline**	**Multicore ST**	**Multicore DT**
Max Clock Frequency (MHz)	31.52	59.6	57.66	54.42	54.42
Execution Time (s)	0.000863	0.000655	0.000235	0.000275	0.000195
Cycles per Instruction	5.108	6.921	2.502	2.760	1.958
Average Latency per Instruction (s)	1.595×10^{-7}	1.211×10^{-7}	4.344×10^{-8}	5.066×10^{-8}	3.592×10^{-8}
Total Registers	1293	1800	4302	8560	8560
Total Combinational Elements	3213	3704	6830	14,843	14,843
Number of Instructions	5409	5409	5409	5429	5429
Speedup (vs. Multicore ST)	0.319	0.420	1.170	1.000	1.410

Table 1: Performance Comparison at Memory Latency = 6 (including avg latency per instruction and speedup)

In Table 1 above you can see how the multicore dual thread is faster than the single thread multicore due to its parallelism. However there is a dip in speed from caches+pipelined to multicore single thread and that is due to the increase in resources used by the multicore single thread. Furthermore, the fact it has two cores causes more resources to be used, hence causing the lower speed. However, this is compensated by the multicore dual thread which is faster than both single thread and caches + pipeline.

Design	LAT = 0 (ms)	LAT = 2 (ms)	LAT = 6 (ms)	LAT = 10 (ms)
Non-Pipelined	0.1943	0.4313	0.8626	1.2938
Pipelined	0.1592	0.3283	0.6554	0.9825
Caches + Pipelined	0.1755	0.1993	0.2347	0.2700
Single Thread Multicore	0.2132	0.2353	0.2753	0.3154
Dual Thread Multicore	0.1402	0.1594	0.1953	0.2325

Table 2: Total Execution Time (in ms) for each processor design at various memory latencies.

In Table 2 you can see how at LAT = 0, pipleine single core is faster than caches + pipelined single core. However, at LAT = 2, the caches start taking an effect causing the caches + pipelined to be faster than the single cycle. In LAT=0 memory is fast enough that the overhead of cache management outweighs any potential speed benefit. However, as memory slows down, caches significantly reduce access time for frequently used data, resulting in lower execution time.

Calculation Formulas and Sources

Max Clock Frequency was found in the system.log file from the 85 degree analysis results for each latency.
Total Registers and Total Combinational Elements were found in the Fitter Summary
Number of instructions was found from system.sim

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

$$\text{Execution Time} = \frac{\text{Instructions}}{\text{Program}} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$\text{Average Clocks per Instruction} = \frac{\text{Total Number of Cycles}}{\text{Total Number of Instructions}}$$

$$\text{Average Latency per Instruction(Single Cycle)} = \frac{\text{Total Execution Time}}{\text{Total Number of Instructions}}$$

$$\text{Average Latency per Instruction} = \frac{\text{CPI}}{\text{Clock Frequency}}$$

4 Conclusion

The results of this project demonstrate the benefits of introducing caches and multicore to a pipelined processor. Adding caches improved performance by reducing the average memory access time, allowing more instructions to complete without stalling due to memory delays. The use of a split cache design for instructions and data prevented structural hazards, improving overall pipeline performance. As expected, the pipelined processor with caches achieved lower CPI and faster execution times compared to the pipelined design without caches.

The multicore design provided additional performance improvements. The multicore system achieved noticeable speedup when compared to single-core execution, proving the effectiveness of parallelism.

Overall, the enhancements of caching and multicore showed clear performance benefits but also increased FPGA resource usage and system complexity. Future improvements could include optimizing the coherence controller or expanding the system to support additional cores. These changes would help maximize the advantages of multicore processing while minimizing the costs.

5 Contributions

Nahom Tadesse: Implemented the dcache logic, icache testbench, the bus logic, the multicore dcache testbench, palgorithm. Muhammad Zohaib Ali: Implemented the dcache testbench, icache logic, the bus testbench, the new dcache (multicore) and lr/sc

However, regardless to say, both parties contributed significantly to each other's work.